

Relatório de Laboratório

Laboratório de Experimentação de Software

Curso	Engenharia de Software
Disciplina	Laboratório de Experimentação de Software
Turno / Período	Noite / 6º
Professor(a)	Danilo Maia
Laboratório	Lab01 - Características de Repositórios Populares + Setup do Kanban
Grupo (trio)	Isabella Dias · Leandro Alencar · Luis Henrique
Link do repositório / GitHub Projects	https://github.com/BGLuis/experimentacao-de-software
Data de entrega	27/08/2026

1. Introdução

Com o crescimento massivo do desenvolvimento de software em plataformas de código aberto, o número de "estrelas" no GitHub consolidou-se como o principal indicador de popularidade de um repositório. No entanto, há uma falta de evidências empíricas consolidadas sobre quais características arquiteturais e sociais realmente impulsionam e mantêm o sucesso desses projetos em larga escala. Compreender o perfil desses repositórios importa profundamente para a Engenharia de Software e para a comunidade open-source, pois permite identificar padrões de engajamento, manutenção sustentável e escolhas tecnológicas que diferenciam projetos ativos de sistemas abandonados.

Neste Lab01, investigamos as características fundamentais dos repositórios mais populares do GitHub por meio da mineração de dados via GraphQL, buscando responder a um conjunto de 6 Questões de Pesquisa (RQs). Para cada questão, estabelecemos as seguintes hipóteses informais antes da etapa de validação dos dados:

RQ01: Idade dos Repositórios Populares

Responsável: Luis

Hipótese Informal: Repositórios mais populares tendem a ser maduros e antigos, pois precisam de tempo para construir uma comunidade sólida e acumular milhares de estrelas.

Métrica: Idade do repositório em anos, calculada a partir da data de criação até a data atual)

Validação e Conclusão: A análise dos dados suporta a hipótese. A mediana de idade dos repositórios na amostra foi de 8,72 anos, com média de 8,33 anos e desvio padrão de 4,32 anos. Isso confirma que a grande maioria dos repositórios de sucesso global são projetos bastante estabelecidos e consolidados ao longo de quase uma década.

RQ02: Contribuição Externa (Pull Requests)

Responsável: Luis

Hipótese Informal: Repositórios populares recebem uma quantidade massiva de contribuições externas através de pull requests, refletindo um alto engajamento e descentralização do desenvolvimento.

Métrica: Total de pull requests aceitas (status merged) por repositório.

Validação e Conclusão: Os resultados confirmam fortemente a hipótese. A mediana de PRs aceitas foi de 134, indicando colaboração externa frequente na maioria absoluta dos projetos. A média foi de 1.273,64 com um desvio padrão muito alto (6.215,67),

demonstrando a presença de repositórios gigantescos que concentram volumes extremos de contribuições externas da comunidade.

RQ03: Frequência de Releases

Responsável: Isabella

Hipótese Informal: Repositórios populares tendem a lançar releases com regularidade, mas alguns podem não usar a ferramenta de releases do GitHub, como listas de curadoria e tutoriais.

Métrica: Número total de releases lançadas por cada repositório.

Validação e Conclusão: Na nova amostra dos Top 1.000 repositórios, a mediana de releases subiu consideravelmente para 39,5, indicando uma regularidade de versionamento muito alta entre a elite do GitHub. Contudo, 28% (280 repositórios) não lançaram nenhuma release, 0 releases. Isso reafirma que uma porção enorme dos projetos mais hypados ainda foca em conteúdos passivos (guias, listas) ou não adota releases formais.

RQ04: Frequência de Atualizações (Push)

Responsável: Isabella

Hipótese Informal: Repositórios populares tendem a ser atualizados muito recentemente, pois projetos altamente ativos atraem e mantêm sua base de usuários.

Métrica: Tempo decorrido (em dias) desde a última atualização de código (utilizei o campo pushedAt para maior precisão).

Validação e Conclusão: Os dados dos 1.000 maiores repositórios comprovam uma atividade diária frenética: 40,5% deles receberam atualizações de código nas últimas 24 horas. A mediana global de inatividade é de apenas 3 dias, contra 50 dias da amostra geral antiga. O outlier mais extremo de abandono nesta elite está há 2.448 dias sem um push sequer, mostrando que a fama pode persistir muito além da manutenção ativa.

RQ05: Linguagens Populares

Responsável: Leandro

Hipótese Informal: Repositórios populares tendem a ser escritos em linguagens também populares na comunidade GitHub, conforme a lista de linguagens mais utilizadas do GitHub Octoverse 2025.

Métrica: Linguagem principal de cada repositório. Como a coleta retorna as linguagens ordenadas por tamanho de código, foi considerada como linguagem principal a primeira linguagem listada.

Validação e Conclusão: Na amostra de 1.000 repositórios, 913 (91,30%) possuem uma linguagem identificada e 87 (8,70%) não possuem linguagem informada. Foram encontrados 702 repositórios (70,20% da amostra; 76,89% entre os identificados) em linguagens presentes no GitHub Octoverse 2025, apoiando a hipótese informal. Python (229), TypeScript (174) e JavaScript (110) foram as linguagens mais frequentes. A distribuição apresentou 43 linguagens distintas e 11 categorias raras, com apenas uma ocorrência; esses casos foram mantidos na análise, pois representam diversidade de linguagens e não dados inválidos.

RQ06: Percentual de Issues Fechadas

Responsável: Leandro

Hipótese Informal: Repositórios populares tendem a possuir uma alta proporção de issues fechadas, pois projetos consolidados geralmente possuem manutenção mais ativa.

Métrica: Razão entre issues fechadas e o total de issues: $\text{issues fechadas} / (\text{issues abertas} + \text{issues fechadas})$. Para tornar alto percentual mensurável, foi adotado o critério operacional de pelo menos 75% de issues fechadas.

Validação e Conclusão: Na amostra de 1.000 repositórios, 957 possuíam issues suficientes para calcular a razão; 43 não possuíam issues e foram excluídos apenas desse cálculo. Não foram encontrados valores ausentes ou inválidos. A mediana da razão de fechamento foi 87,62% e 678 repositórios (70,85% dos casos válidos) alcançaram pelo menos 75% de issues fechadas, apoiando a hipótese informal. Foram identificados 40 outliers pela regra de 1,5 vezes o intervalo interquartil, todos na cauda inferior da distribuição; eles foram preservados para representar projetos com padrão de tratamento de issues diferente.

2. Contexto

ORIENTAÇÃO: Situe o leitor no cenário do estudo. Primeiro, o contexto acadêmico: em qual momento do semestre este laboratório se encontra e como ele se conecta aos anteriores (ex.: “este é o Lab04, que consome os dados de mineração do Lab03 e os snapshots do Kanban mantidos desde o Lab01”). Segundo, o contexto do objeto de estudo em si: o que exatamente está sendo medido (os 1.000 repositórios mais populares do GitHub — Lab01; o processo de resolução de katas com e sem IA — Lab02; repositórios com CI/CD via GitHub Actions — Lab03; o board Kanban do próprio grupo — Lab04/Lab05). Cite aqui referências conceituais relevantes usadas como base teórica (ex.: o livro *Accelerate*, de Forsgren, Humble & Kim, para métricas DORA; o método GQM de Basili, Caldiera & Rombach para o meta-laboratório; o índice usado para “linguagens mais populares” no Lab01 — TIOBE, GitHub ou GitHub Octoverse, mantendo a mesma fonte do início ao fim).

[conteúdo do grupo — substituir este texto]

3. Metodologia

ORIENTAÇÃO: Esta é a seção mais longa do relatório e a que mais evidencia o trabalho real do grupo. Ela tem seis subseções — as cinco primeiras cobrem principalmente os 70% do enunciado; a última (Inovações) é onde os 30% de contribuição própria do grupo devem ficar explícitos e fáceis de identificar na correção.

3.1 Principais Desafios

ORIENTAÇÃO: Relate as dificuldades técnicas e metodológicas reais enfrentadas pelo grupo — não uma lista de trivialidades já resolvidas, e sim decisões difíceis de fato. Exemplos típicos, conforme o laboratório: limite de taxa (rate limit) da API do GitHub ao consultar milhares de repositórios ou workflow runs (Lab01/Lab03); paginação de grandes volumes de dados; ausência de histórico de mudança de status consultável via API no GitHub Projects, exigindo snapshots manuais recorrentes (todos os laboratórios); dificuldade de padronizar katas de dificuldade equivalente e evitar memorização de soluções pela IA (Lab02); ambiguidade na definição operacional de uma métrica, como lead time (Lab03); dados incompletos ou repositórios sem GitHub Actions habilitado (Lab03).

[conteúdo do grupo — substituir este texto]

3.2 Tomadas de Decisão

ORIENTAÇÃO: Documente as decisões metodológicas do grupo e o raciocínio (trade-off) por trás de cada uma — não apenas a escolha final. Exemplos que os enunciados pedem explicitamente: o limite de WIP definido para a coluna Doing e sua justificativa (obrigatório em todo laboratório); qual assistente de IA foi usado e por quê, e como se garantiu o mesmo tratamento em todos os trials (Lab02); qual definição operacional de métrica foi adotada quando o enunciado permite variação, mantendo-a consistente para toda a amostra (ex.: lead time no Lab03); critério de inclusão/exclusão de repositórios na amostra; linguagem de programação escolhida em função da ferramenta de métricas estáticas disponível (CK exige Java; Radon para Python).

[conteúdo do grupo — substituir este texto]

3.3 Etapas

ORIENTAÇÃO: Descreva o processo de desenvolvimento em sprints, seguindo a estrutura do enunciado (ex.: Lab0XS01, S02, S03 + Relatório Final), com o que foi efetivamente entregue em cada uma e quem (qual integrante) foi responsável por qual parte — a correção do professor é feita a partir do board (GitHub Projects), então a divisão aqui deve refletir os Assignees reais das Issues, não uma divisão apenas narrativa. Inclua também a subseção “Configuração do processo” exigida em todos os laboratórios: as colunas do board (mínimo Backlog → To Do → Doing → Review → Done), a política de limite de WIP em uso, e uma captura de tela (print) do board ao final do laboratório, mostrando o fluxo real de trabalho do grupo.

[Tabela ou linha do tempo com Sprint | Entregas | Responsável(is) | Issues (nº)]

Sugestão: insira aqui o print do quadro Kanban (GitHub Projects) mencionado na orientação acima.

3.4 Ferramentas

ORIENTAÇÃO: Liste as ferramentas usadas na coleta, processamento e análise de dados — sejam específicas (nome e versão quando relevante), não genéricas. Exemplos conforme o laboratório: GraphQL e/ou REST API do GitHub para mineração (Lab01/Lab03 — bibliotecas de terceiros para consulta à API não são permitidas, o script deve ser próprio do grupo); Python/Pandas para manipulação de dados; Matplotlib/Seaborn ou Plotly/Dash/Streamlit para visualização; CK, PMD ou Radon para métricas estáticas de código (Lab02); testes estatísticos como o de Wilcoxon para amostras pareadas (Lab02); ferramenta de BI (Power BI, Tableau, Looker Studio) caso o grupo não opte pelo dashboard em código (Lab04). Inclua também a ferramenta de processo, obrigatória em todos os laboratórios: GitHub Projects (v2), com o link do repositório/board do grupo.

[conteúdo do grupo — substituir este texto]

3.5 Tabela de Métricas

ORIENTAÇÃO: Construa uma tabela relacionando cada Questão de Pesquisa à métrica correspondente, sua definição operacional exata (a fórmula ou regra de cálculo — não basta o nome) e a ferramenta/fonte usada para coletá-la. Isso é o que garante que o laboratório seja reproduzível por outro grupo. A primeira linha abaixo é um exemplo ilustrativo (baseado no Lab01); substitua pelas RQs e métricas do seu laboratório.

RQ	Métrica	Definição Operacional	Unidade	Ferramenta / Fonte
RQ01 (exemplo)	Idade do repositório	Data atual – data de criação do repositório	Dias	Script GraphQL (API do GitHub)

3.6 Inovações Propostas pelo Grupo (30% da nota)

ORIENTAÇÃO: O enunciado do laboratório corresponde a 70% da exigência da disciplina. Os outros 30% dependem de uma contribuição original do grupo, que deve estar claramente identificada aqui — não diluída no restante do texto — para facilitar a correção. Escolha uma ou mais frentes de inovação, entre: (a) uma nova Questão de Pesquisa, além das do enunciado; (b) uma métrica ou variável adicional, não pedida no enunciado; (c) uma mudança de arquitetura/ferramenta de coleta (ex.: paralelizar a coleta, usar cache, trocar de biblioteca de visualização); (d) uma metodologia alternativa ou complementar (ex.: um teste estatístico adicional, uma segmentação diferente da amostra, uma técnica de controle de ameaça à validade não exigida pelo enunciado). Para cada inovação escolhida, explique o que foi feito, por que o grupo considerou relevante, e onde o resultado dela aparece nas seções de Resultados/Discussão e na Conclusão — inovação sem resultado discutido não conta como contribuição efetiva.

[conteúdo do grupo — substituir este texto]

4. Resultados

4.1 Coleta de Dados

ORIENTAÇÃO: *Relate o volume final de dados efetivamente coletado e analisado — não apenas o volume-alvo do enunciado. Informe: quantos itens restaram após os filtros de qualidade (ex.: dos 1.000 repositórios buscados, quantos tinham dados completos; dos repositórios candidatos, quantos de fato usavam GitHub Actions — Lab03); o período coberto pela coleta; quantos trials/execuções foram concluídos dentro do tempo (Lab02); quantos snapshots do Kanban estão disponíveis e desde quando (Lab04/Lab05); outliers ou dados ausentes identificados, e como foram tratados (removidos, mantidos e discutidos à parte, etc.).*

[conteúdo do grupo — substituir este texto]

4.2 Visualização Gráfica

ORIENTAÇÃO: *Para cada Questão de Pesquisa (do enunciado e das RQs de inovação do grupo), inclua ao menos uma visualização que a responda diretamente, com a pergunta enunciada em texto imediatamente antes do gráfico correspondente, eixos nomeados com clareza e a medida de tendência central adequada indicada (mediana costuma ser preferível a média quando há outliers ou distribuição assimétrica — comum em dados de repositórios de software). Use o tipo de gráfico adequado ao tipo de pergunta, conforme a tabela abaixo, e explicita no texto os valores-chave que aparecem no gráfico (não deixe o leitor “adivinhar” o número a partir da figura).*

Tipo de pergunta / dado	Gráfico recomendado
Comparar uma métrica entre categorias (ex.: linguagem, benchmark DORA)	Barras (ranking) — ordenadas por valor, não alfabeticamente
Comparar dois tratamentos no mesmo grupo (ex.: com IA vs. sem IA)	Boxplot pareado ou gráfico de pontos conectados (before/after)
Distribuição de uma métrica numérica (ex.: idade dos repositórios)	Histograma ou boxplot
Relação entre duas métricas numéricas (ex.: RQ07 do Lab01, RQ05 do Lab03)	Gráfico de dispersão (scatter plot)
Evolução ao longo do tempo (ex.: cycle time por sprint)	Linha, com um ponto por sprint/snapshot
Composição/fluxo do Kanban ao longo do tempo (Cumulative Flow Diagram)	Área empilhada (uma camada por coluna do board)
Proporção de categorias (ex.: % de issues fechadas)	Barra única 100% ou barras simples — evite pizza com muitas fatias

[Insira aqui os gráficos do grupo, um por RQ, cada um precedido da pergunta que ele responde]

4.3 Discussão

ORIENTAÇÃO: *Para cada RQ (do enunciado e das RQs de inovação do grupo), compare explicitamente a hipótese informal levantada na Introdução com o resultado efetivamente obtido — hipótese confirmada, refutada, ou parcialmente confirmada, e por quê. Quando houver teste estatístico (ex.: Wilcoxon no Lab02), reporte o valor obtido e interprete o que ele significa em linguagem acessível, não apenas o número bruto. Discuta as ameaças à validade específicas do laboratório (ex.: efeito de aprendizado entre katas e risco de memorização pela IA — Lab02; diferença de dificuldade entre laboratórios distintos confundindo a tendência de cycle time — Lab05; lacunas nos snapshots do Kanban — Lab05). Finalize relacionando o que*

as inovações do grupo (seção 3.6) acrescentaram: elas confirmaram, contradisseram ou aprofundaram o que os 70% do enunciado já mostravam?

[conteúdo do grupo — substituir este texto]

5. Conclusão

ORIENTAÇÃO: Sintetize, em poucos parágrafos, as respostas a todas as RQs (enunciado + inovação do grupo), sem repetir números já discutidos em detalhe — o objetivo aqui é a mensagem final, não os dados brutos. Aponte as principais limitações do estudo (tamanho de amostra, ameaças à validade não mitigadas, período de coleta). Quando o enunciado pedir explicitamente uma postura de consultoria (caso do Lab05, que pede recomendações de melhoria de processo “como se o grupo fosse consultoria para um time real”), inclua recomendações objetivas e acionáveis, não genéricas. Encerre indicando o que o grupo faria diferente com mais tempo ou recursos, e quais das inovações propostas (30%) valeriam a pena expandir em um trabalho futuro.

[conteúdo do grupo — substituir este texto]

5. Referências

- GitHub Octoverse. "A new developer joins GitHub every second as AI leads TypeScript to #1". 2025. Disponível em: <https://github.blog/news-insights/octoverse/octoverse-a-new-developer-joins-github-every-second-as-ai-leads-typescript-to-1/>